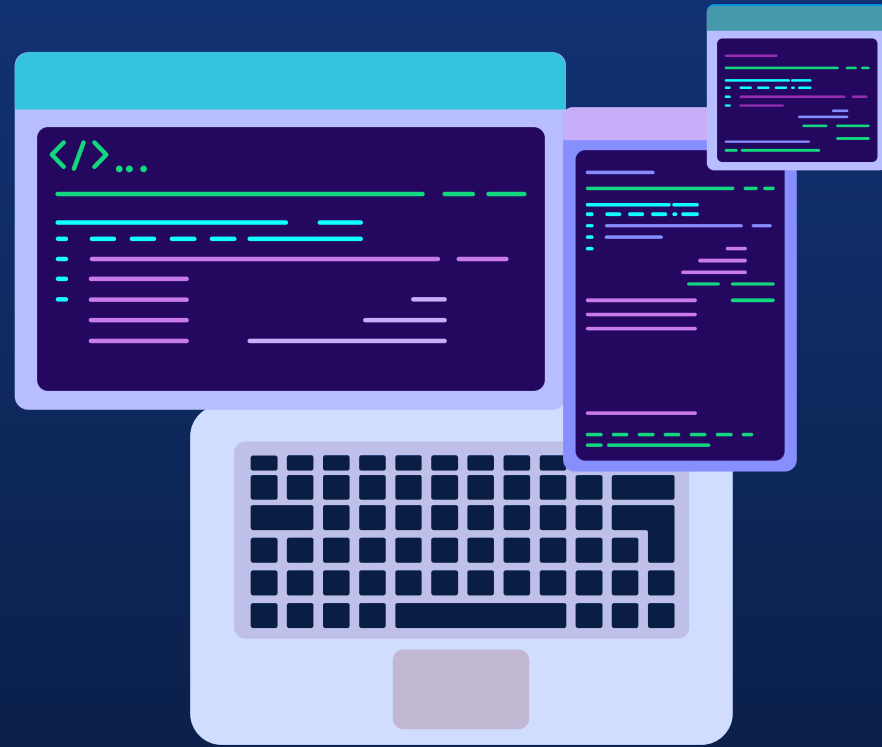


Administration automatisée Powershell



Introduction à powershell

- Créer en 2006
- Intégré à windows XP en package optionnel puis windows 7 en natif
- Actuellement en V7.5
- Fonctionne sur Framework .NET
- Permet d'administrer windows entièrement (AD DNS etc etc)
- Gère les objets fonctions exceptions modules etc
- Editeur powershell ISE (VS Code maintenant)
- Option shell avec wsl



Etape de création

01

Cahier des charges

Etablissement des objectifs et fonctionnalités

02

Coder un premier jet

Test des endpoints

03

Nettoyage du code

fonctions et sorties inutiles, nommage correct des variables

04

Optimisation du code

Analyse des points dur du script

05

Ordonnancement

Mise en place d'une récurrence

06

Documentation

Documenter le code et ça façon de fonctionner



“N’importe quel idiot peut écrire du
code qu'un ordinateur peut
comprendre. Les bons
programmeurs écrivent du code
que les humains peuvent
comprendre.”



—Martin Fowler



Les variables

Portée des variables

- Peu ou pas utilisé en powershell, elles peuvent avoir un intérêt uniquement dans des cas particulier de sécurité

Protections des variables

- A Utiliser à chaque passage dans une chaine de caractère et chaque sortie
ex:
 - `$nom="toto" → Write-Host "je m'appelle $($nom) ou ${nom}"`
 - `$user=Get-Aduser -filter 'cedric' -server "$($server)" → Write-Host "nom mail est $ ($user.mail)"`

Savoir le type de variable

- `gm -i $var`
retourne le type et les méthodes et propriétés associés

Type de variable

int	string	arraylist	hash	object	secure string	float	boolean	date time	custom
-----	--------	-----------	------	--------	---------------	-------	---------	-----------	--------



Les strings

Echappement

Le caractère d'échappement est ` (ALT GR + 7)

Simple et double quote

Simple quote pas de prise en compte des caractères spéciaux

Double quote prise en charges des variables et CS

Généralité

Permet les méthodes split replace trim ...

Stocké comme un tableau de lettre

```
$machaine="ceci est un string"
for((i=0; i< ${machaine%$*} )); do echo ${machaine:i:1}
done
```

c
e
c
i

e
s
t

u
n

s
t
r
i
n
g



Les Arrays et Hashtable

Array

Mode objet \$myarray=@() \$myarray+="test"	Mode array \$myarray=[System.Collections.ArrayList]@() \$myarray.add("test") \$myarray.RemoveAt(0)
--	---

Hashtable
\$myhashtable=@{ }
\$myhashtable.add("mykey","myvalue")
\$myhashtable.item("mykey") ⇒ renvoie la value associée



If condition et opérateurs

Structure

```
if(){  
}elseif(){  
}else{  
}
```

multiple conditions

- and
- or
- not
- xor

Equality

- `-eq`, `-ieq`, `-ceq` - equals
- `-ne`, `-ine`, `-cne` - not equals
- `-gt`, `-igt`, `-cgt` - greater than
- `-ge`, `-ige`, `-cge` - greater than or equal
- `-lt`, `-ilt`, `-clt` - less than
- `-le`, `-ile`, `-cle` - less than or equal

Matching

- `-like`, `-ilike`, `-clike` - string matches wildcard pattern
- `-notlike`, `-inotlike`, `-cnotlike` - string doesn't match wildcard pattern
- `-match`, `-imatch`, `-cmatch` - string matches regex pattern
- `-notmatch`, `-inotmatch`, `-cnotmatch` - string doesn't match regex pattern

Replacement

- `-replace`, `-ireplace`, `-creplace` - replaces strings matching a regex pattern

Containment

- `-contains`, `-icontains`, `-ccontains` - collection contains a value
- `-notcontains`, `-inotcontains`, `-cnotcontains` - collection doesn't contain a value
- `-in` - value is in a collection
- `-notin` - value isn't in a collection

Type

- `-is` - both objects are the same type
- `-isnot` - the objects aren't the same type

Les boucles

boucle sur un objet, array, ou hashtable

```
foreach($item in $object){  
  ...  
}
```



boucle via un indice

```
for ($i; $i -le $test.length;  
$i++){  
  write-host "$($test[$i])"  
}
```

```
while($val -ne 3)  
{  
  $val++  
  Write-Host $val  
}
```

do {<statement list>} while (<condition>)

do {<statement list>} until (<condition>)



Les sorties

Write-Host → sortie dans le prompt

Write-Error → sortie dans le flux d'erreur

Write-Debug → sortie dans la console

Write-Information → sortie dans flux standard

Write-Output → sortie dans la console

Write-Progress → affiche une barre de progression dans le prompt*

Write-Verbose → affiche des messages en plus sous certaines conditions-

Write-Warning → affiche le message en warning

A utiliser suivant le cas la sortie voulu

Les options permettent de changer la couleur du texte ou le fond par exemple



Le logging

Important : La partie logging peut considérablement réduire la vitesse d'un script

Ajouter du texte dans un fichier

Add-content monfichier.txt -value "logging"

D'autres moyens sont possible pour accélérer cette partie d'écriture

Permettre un mode debug à la demande

```
Administrator: Windows PowerShell
PS C:\> C:\Users\administrator\Desktop\MSG_UTILITY.ps1
Type Your Message Here: Hi, updates are being installed on your computer, please wait and don't reboot.
Type Computer Name Here: c:\name.txt
Type Time Here: 5
Sending Message to NOC-Pc-21.....
Message Successfully Sent to NOC-Pc-21
Sending Message to NOC-Pc-19.....
Message Successfully Sent to NOC-Pc-19
NOC-PC-25 is not Reachable...
Sending Message to NOC-Pc-22.....
Message Successfully Sent to NOC-Pc-22
Sending Message to NOC-PC-17.....
Message Successfully Sent to NOC-PC-17
NOC-PC-18 is not Reachable...
Start at 4:39:13 PM End At 4:39:20 PM Took About 0 Minutes 7 Seconds
Total 6 Computer Processed, 2 computers were offline. The list is stored in Not_Reachable PCs.txt
PS C:\> _
```

La performance

Mettre en place des jalons de performance

```
$Step=0  
$StepTime = [Diagnostics.Stopwatch]::StartNew()  
Write-Host "Step $($Step) - Initialization : $($StepTime.Elapsed.TotalSeconds) seconds"  
$StepTime.Restart()  
$Step++
```

Vérification des perfs CPU et RAM en même temps

Les entrants

```
param (  
    [Parameter(Mandatory = $false, ParameterSetName="Job")]  
    [ValidateLength(1,20)]  
    [string]  
    $Jobname,  
  
    [Parameter(Mandatory = $true)]  
    [ValidateSet("ON_HOLD", "RELEASED", "WAITING", "RUNNING", "CANCELED", "FINNED_OK", "FAILED", "MISSED", "FINISHED")]  
    [string]  
    $Status,  
  
    [Parameter(Mandatory = $False)]  
    [ValidateScript(  
        {$_ -ge (Get-Date).Year})]  
    [String]$Date = (get-date -format yyyy)  
)
```

monscript.ps1 -Job 'folder_creation' -Status 'ON_HOLD' -Date 2029

Les secrets et credentials

Credential : domain/user et password

- Avec un passfile et une clé
 - chiffrement du password dans un fichier et stockage de la clé ailleurs
- Passage d'un credential
 - Préfait
 - Via la commande Get-credential
- Stockage via le vault windows
 - accessible uniquement depuis la session qui a stocké le credential



WinRm

Port : 5986 en chiffré et 5985 en non chiffré

Vérifier les ports dans le firewall

Permet l'exécution de commande et script à distance

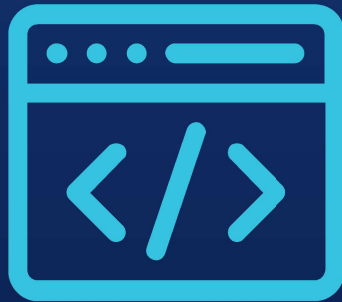
Authentification via Default, Basic, Negotiate, NegotiateWithImplicitCredential, Credssp, Digest, Kerberos

Création d'une session powershell :

- New-pssession

Execution à distance

- invoke-command -session <varsession> -scriptblock{<commande> }

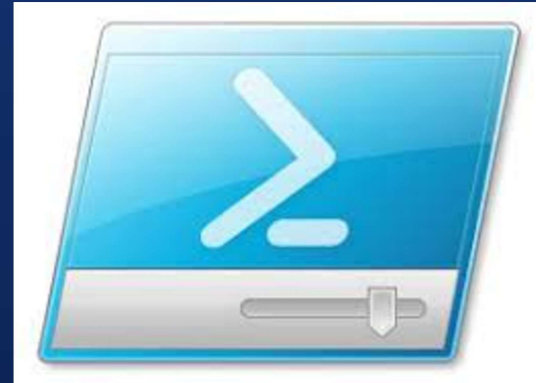


Powershell ISE

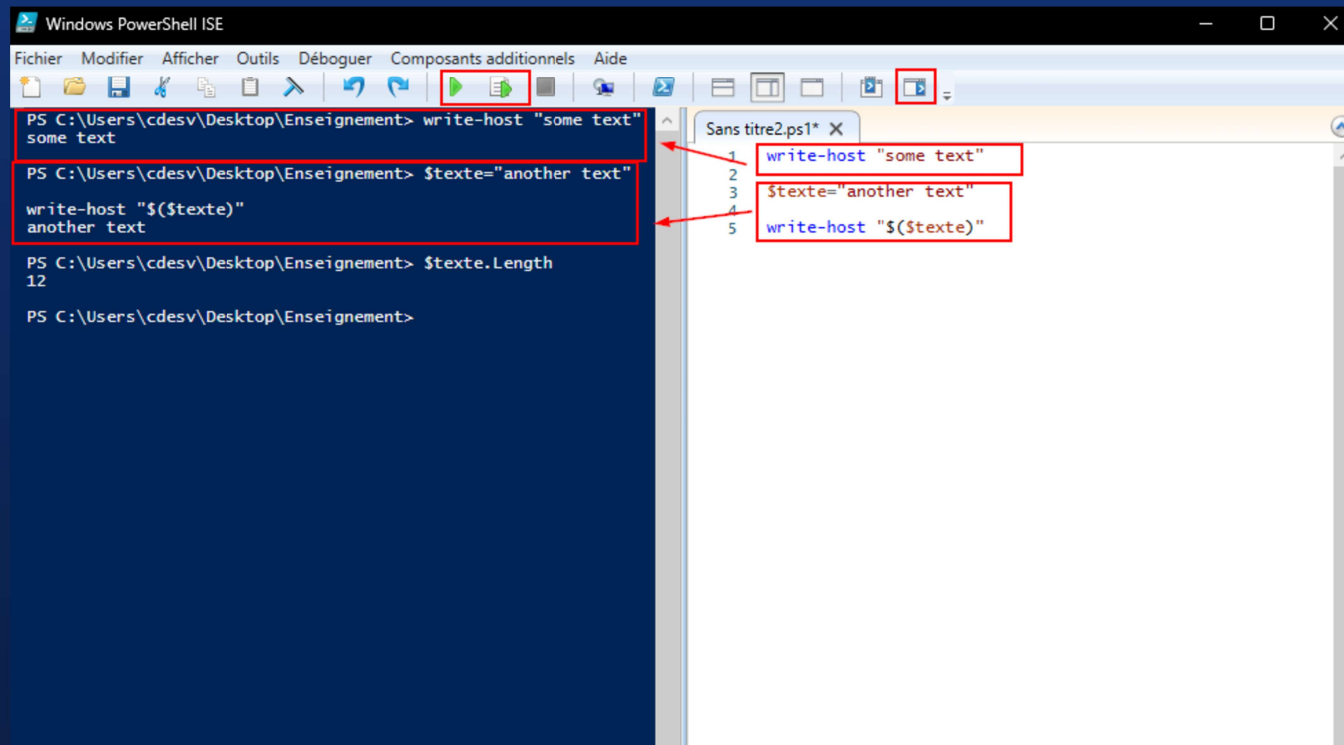
- Interface et définition
- Utilisation

Interface

- Fourni par Windows
- Debug de code
- Aide
- Auto-completion
- Pris en charge jusqu'à PS V5.1
- Remplacer par visual code



Utilisation



```
Windows PowerShell ISE
Fichier Modifier Afficher Outils Débugger Composants additionnels Aide

PS C:\Users\cdesv\Desktop\Enseignement> write-host "some text"
some text

PS C:\Users\cdesv\Desktop\Enseignement> $texte="another text"

write-host "$($texte)"
another text

PS C:\Users\cdesv\Desktop\Enseignement> $texte.Length
12

PS C:\Users\cdesv\Desktop\Enseignement>

Sans titre2.ps1* X
1 write-host "some text"
2
3 $texte="another text"
4
5 write-host "$($texte)"
```

Utilisation

The screenshot illustrates the usage of the `Write-Host` command in Visual Studio. It shows a PowerShell console window with the following commands and output:

```
PS C:\Users\cdesv\Desktop\Enseignement> Write-Host -BackgroundColor black -ForegroundColor green -Object $texte
another text
PS C:\Users\cdesv\Desktop\Enseignement> show-command
PS C:\Users\cdesv\Desktop\Enseignement> show-command 'write-host'
```

The `Write-Host` command is highlighted in red in the console. A red box labeled '2' points to the `Write-Host` command in the PowerShell console. A red box labeled '3' points to the `Write-Host` command in the `Sans titre2.ps1` script file.

The `Write-Host` dialog box is also shown, with the `Write-Host` command selected in the `Paramètres pour « Write-Host » :` section. A red box labeled '3' points to the `Write-Host` command in the dialog box.

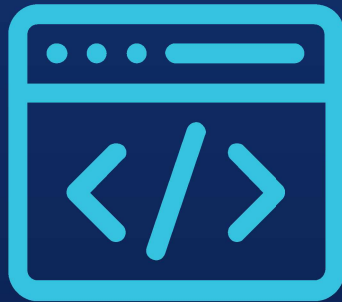
The `Write-Host` dialog box contains the following settings:

- `BackgroundColor`: black
- `ForegroundColor`: green
- `NoNewline`: ☐
- `Object`: \$texte
- `Separator`:

The `Paramètres communs` section is also visible, with the following settings:

- `Debug`: ☐
- `ErrorAction`:
- `ErrorVariable`:
- `InformationAction`:
- `InformationVariable`:

The `Write-Host` dialog box has buttons for `Exécuter`, `Copier`, and `Annuler`.



Erreur de Code

- Fonction dans fonction
- Script qui appelle des scripts qui appelle des scripts
- Trop de commentaire dans le code
- Les noms de variable qui veulent rien dire
- Les password dans les scripts
- Les abréviations de nom de fonction

Fonction dans fonction

Script dans script

```
#####
# DotSourcing :
#####
$Script:dirScript = $PSScriptRoot

# fonctions de base
. $Script:dirScript\..\includes\FonctionsBase.ps1
. $Script:dirScript\..\includes\Env.ps1
. $Script:dirScript\..\includes\CMDB.ps1
. $Script:dirScript\..\includes\Nagios.ps1

##### LOGS + infos diverses #####
# verbosité :
|
| 0 : error
| 1 : warning
| 2 : verbose
| 3 : debug
| 4 : stop en cas d'erreur #>
AjusteVerbosity 1

#####
# Variables de script :
#####

# Fichier de log traçant le déroulement du script
$script:LogFile="$Script:dirScript\logs\${Title}.log"
Write-Debug "LogFile=[$script:LogFile]"
```

Trop de commentaire dans le code

```
#### BARRE PROGRESSION Script ID=1 ####
$ScriptStep = 0
$TotalScriptSteps = 6 # Récupération des données, CMDB, VMWARE, Nagios, AD_GDC, Synthèse
$ScriptStep++
#Write-Progress -id 1 -Activity "[Script] Etapes " -PercentComplete ($ScriptStep / $TotalScriptSteps * 100) -CurrentOperation "Récupération des données" -Status "Item $ScriptStep sur $TotalScriptSteps"

# Récupération des listes
# On récupère la liste des machines du Workplace dans la CMDB
WriteToHostAndLog -log $script:LogFile -type ' STEP ' -msg $("récupération CMDB")
$script:listeCMDB = get-AllNSCComputersInfos -idProgressBarre 2

# $script:listeCMDB = $script:listeCMDB |?($_.root_iscandidatefordeletion -ne "True")

####-> NETTOYAGE DES INFOS MERDRIQUES REMONTEES DANS LA CMDB
####-> # Correction du nom d'une machine qui a le DNS : localhost.localdomain
# VM totalement déconnectée, arrêtée, et supprimée de la CMDB
# ($script:listeCMDB |?($_.global_id -eq "XXXXXXXXXX")).name = "XXXXXXXXXX"
####-> # Correction du nom d'une machine qui a le DNS : localhost.localdomain
# ($script:listeCMDB |?($_.global_id -eq "XXXXXXXXXX")).name = "XXXXXXXXXX"
# ($script:listeCMDB |?($_.global_id -eq "XXXXXXXXXX")).name = "XXXXXXXXXX"
####-> # Correction du nom d'une machine qui a le DNS : localhost.localdomain
# ($script:listeCMDB |?($_.global_id -eq "XXXXXXXXXX")).name = "XXXXXXXXXX"
####-> # Correction du nom d'une machine qui a le DNS : localhost.localdomain
# ($script:listeCMDB |?($_.global_id -eq "XXXXXXXXXX")).name = "XXXXXXXXXX"
####-> # Correction du nom d'une machine qui a le DNS : localhost.localdomain
# ($script:listeCMDB |?($_.global_id -eq "XXXXXXXXXX")).name = "XXXXXXXXXX"
####-> # Correction du nom d'une machine qui a le DNS : localhost.localdomain
# ($script:listeCMDB |?($_.global_id -eq "XXXXXXXXXX")).name = "XXXXXXXXXX"
```

Les noms de variable qui veulent rien dire

```
$maxhauteur= Read-Host "Quelle est sa hauteur ?"

$maxlongueur= Read-Host "Quelle est sa longueur?"

$i=0
for($i;$i -lt $maxhauteur;$i++){
    $j=0
    for($j;$j -lt $i;$j++){
        Write-Host " " -NoNewline
    }
    $h=0
    for($h;$h -lt $maxlongueur;$h++){
        if($h -eq $maxlongueur-1){
            Write-Host ""
        }else{
            Write-Host "" -NoNewline
        }
    }
}

for($i;$i -ge 0;$i--){
    $j=0
    for($j;$j -lt $i;$j++){
        Write-Host " " -NoNewline
    }
    $h=0
    for($h;$h -lt $maxlongueur;$h++){
        if($h -eq $maxlongueur-1){
            Write-Host ""
        }else{
            Write-Host "" -NoNewline
        }
    }
}
}
```




Les abréviations de nom de fonction

|%{} pour foreach sur un objet

|?{} pour un where object

```
... | Select-Object -Expand Name | ?{  
  Test-Connection $_ -Quiet  
} | % {  
  Get-WmiObject -ComputerName $_  
}
```

```
$machines_name= ... | Select-Object -ExpandProperty Name  
foreach($machine in $machines_name){  
  if(Test-NetConnection $machine -Quiet){  
    Get-WmiObject -ComputerName $machine  
  }  
}
```

